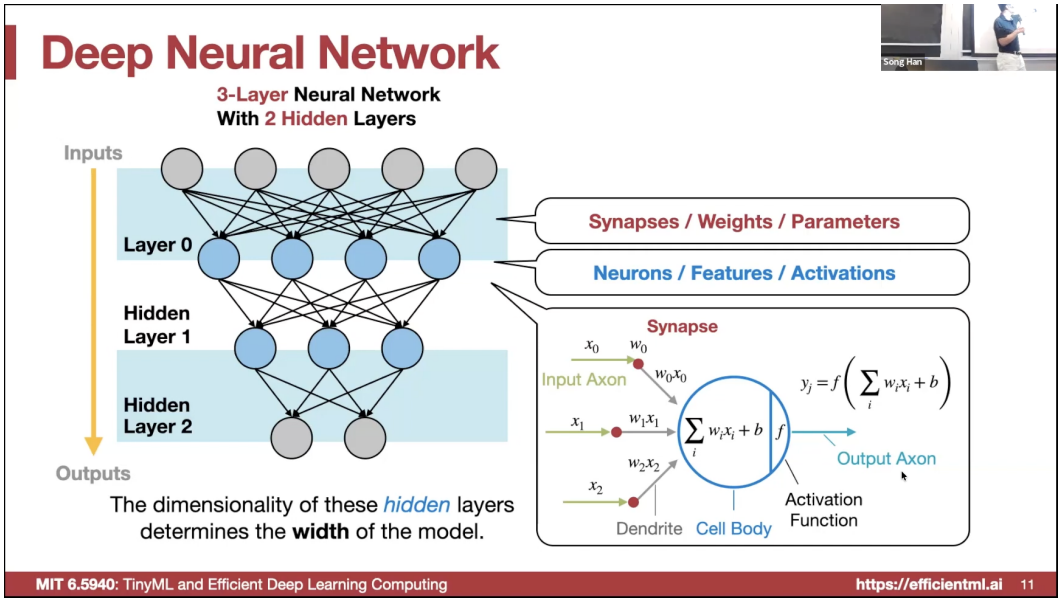
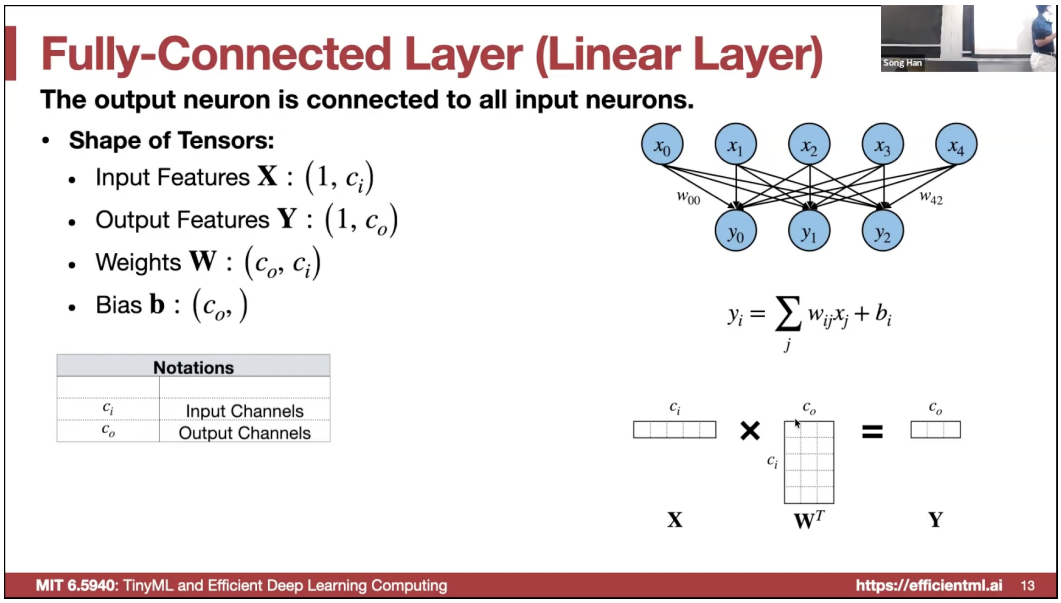
# Neural Network from Scratch with NumPy

**Hariprashad Ravikumar**

## 1 TinyML and Efficient AI Computing

### 1.1 Lecture 2 - Basics of Neural Networks



# 2 Hari's personal note on Neural Networks

## A Simple Neural Network

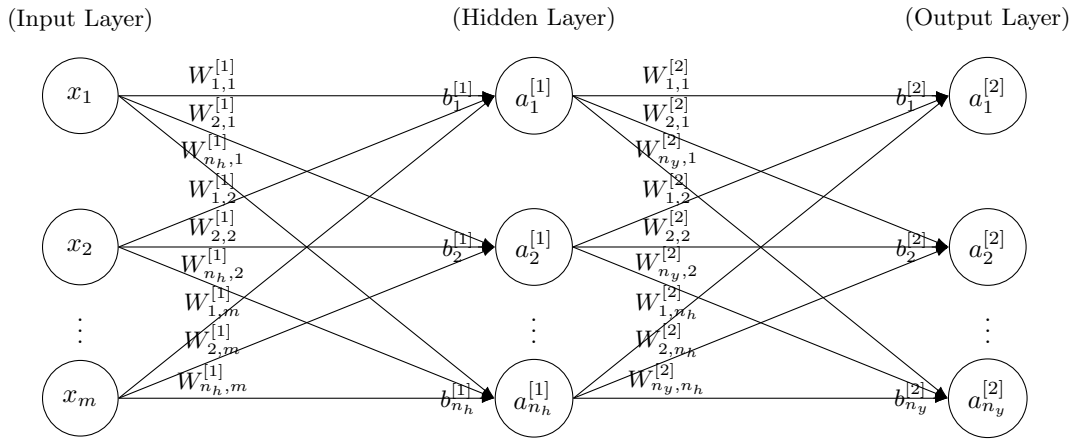



Figure 1: Simple Two Layer Neural Network (NN)

- $m$ = number of training examples.
- $X \in \mathbb{R}^{n_x \times m}$: input data, each column $x^{(i)}$ is one example.
- $Y \in \mathbb{R}^{n_y \times m}$: output data
- For example, in MNIST, each image is $28 \times 28 = 784$ pixels; we want have output of 10 row vector $(0, 1, 2, \ldots, 9)$. In this case, $n_x = 784$, $n_y = 10$. For simiplicity let's choose the number of nodes in the hidden layer as $n_h = 10$

$$A^{[0]} = X \in \mathbb{R}^{784 \times m} \tag{1}$$
$$W^{[1]} \in \mathbb{R}^{10 \times 784} \tag{2}$$
$$b^{[1]} \in \mathbb{R}^{10 \times m} \tag{3}$$

- **Layer 1 (hidden)**:

$$W^{[1]} \in \mathbb{R}^{n_h \times n_x}, \quad b^{[1]} \in \mathbb{R}^{n_h \times m},$$
$$Z^{[1]} = W^{[1]} X + b^{[1]}, \quad A^{[1]} = g\big(Z^{[1]}\big),$$

  where $g(\cdot)$ is an element-wise activation (e.g., ReLU or tanh).

- **Layer 2 (output)**:

$$W^{[2]} \in \mathbb{R}^{n_y \times n_h}, \quad b^{[2]} \in \mathbb{R}^{n_y \times m},$$
$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}, \quad A^{[2]} = \mathrm{softmax}\big(Z^{[2]}\big),$$

  so that each column $A^{[2]}_{:,i}$ is a probability vector over $n_y$ classes for example $i$.

We train with the average cross-entropy loss:

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{n_y} Y_{k,i} \log\big(A^{[2]}_{k,i}\big). \tag{4}$$

Our goal is to compute the gradients

$$\frac{\partial J}{\partial W^{[2]}}, \quad \frac{\partial J}{\partial b^{[2]}}, \quad \frac{\partial J}{\partial W^{[1]}}, \quad \frac{\partial J}{\partial b^{[1]}},$$

so that we can update parameters by (for example) gradient descent. Below is the full chain-rule derivation.

# 1. Forward Propagation

### 1.1 Input layer → Hidden layer

$$Z^{[1]} = W^{[1]}\,X + b^{[1]} \;\in\; \mathbb{R}^{n_h \times m}, \quad A^{[1]} = g\big(Z^{[1]}\big) \;\in\; \mathbb{R}^{n_h \times m}.$$

Here $g$ is applied element-wise (e.g., ReLU: $g(z) = \max(0, z)$, or $\tanh(z)$, etc.).

### 1.2 Hidden layer → Output layer

$$Z^{[2]} = W^{[2]}\,A^{[1]} + b^{[2]} \;\in\; \mathbb{R}^{n_y \times m}, \quad A^{[2]} = \mathrm{softmax}\big(Z^{[2]}\big) \;\in\; \mathbb{R}^{n_y \times m}.$$

Concretely, for each example $i$ and each class $k$,

$$A^{[2]}_{k,i} \;=\; \frac{\exp\big(Z^{[2]}_{k,i}\big)}{\sum_{j=1}^{n_y} \exp\big(Z^{[2]}_{j,i}\big)}, \quad k = 1, \ldots, n_y.$$

### 1.3 Compute loss

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{n_y} Y_{k,i}\, \log\big(A^{[2]}_{k,i}\big).$$

# 2. Backpropagation: Output-Layer Gradients

Because we use softmax plus cross-entropy, there is a well-known simplification:

$$\frac{\partial J}{\partial Z^{[2]}} \;=\; A^{[2]} \;-\; Y \quad \in\; \mathbb{R}^{n_y \times m}. \tag{5}$$

More explicitly, for each class $k$ and example $i$,

$$\big[dZ^{[2]}\big]_{k,i} \;=\; A^{[2]}_{k,i} \;-\; Y_{k,i}.$$

We denote this matrix by

$$dZ^{[2]} \;\equiv\; A^{[2]} - Y.$$

## 2.1 Gradient w.r.t. $W^{[2]}$

From

$$Z^{[2]} = W^{[2]}\,A^{[1]} + b^{[2]},$$

treating $A^{[1]}$ as constant,

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial Z^{[2]}}\,\frac{\partial Z^{[2]}}{\partial W^{[2]}}.$$

In matrix form, since $Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$,

$$\frac{\partial J}{\partial W^{[2]}} = \frac{1}{m}\,\big(dZ^{[2]}\big)\,\big(A^{[1]}\big)^{T} \;\in\; \mathbb{R}^{n_y \times n_h}. \tag{6}$$

(The factor $1/m$ comes from averaging over $m$ examples.)

## 2.2 Gradient w.r.t. $b^{[2]}$

Because $b^{[2]}$ is broadcast-added to each column of $W^{[2]}A^{[1]}$,

$$\frac{\partial J}{\partial b^{[2]}} = \frac{1}{m} \sum_{i=1}^{m} \big[dZ^{[2]}\big]_{:,\,i} = \frac{1}{m}\, dZ^{[2]}\,\mathbf{1}_m,$$

where $\mathbf{1}_m \in \mathbb{R}^m$ is a column-vector of all ones. In practice:

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[2]}_{:,\,i} \;\in\; \mathbb{R}^{n_y \times 1}. \tag{7}$$

### 2.3 Gradient w.r.t. $A^{[1]}$ (propagate to previous layer)

Since $Z^{[2]} = W^{[2]}\,A^{[1]} + b^{[2]}$,

$$dA^{[1]} = \frac{\partial J}{\partial A^{[1]}} = \big(W^{[2]}\big)^T\,dZ^{[2]} \quad \in \mathbb{R}^{n_h \times m}. \tag{8}$$

We often call this intermediate term $dA^{[1]} = \big(W^{[2]}\big)^T\,dZ^{[2]}$.

## 3. Backpropagation: Hidden-Layer Gradients

At layer1, we have

$$Z^{[1]} = W^{[1]}\,X + b^{[1]}, \quad A^{[1]} = g\big(Z^{[1]}\big),$$

where $g$ is the activation (e.g., ReLU or tanh). We already computed

$$dA^{[1]} = \big(W^{[2]}\big)^T\,dZ^{[2]} \;\in\; \mathbb{R}^{n_h \times m}.$$

### 3.1 Compute $dZ^{[1]}$

By the chain rule,

$$dZ^{[1]} = \frac{\partial J}{\partial Z^{[1]}} = dA^{[1]} \circ g'\big(Z^{[1]}\big), \tag{9}$$

where "$\circ$" denotes element-wise (Hadamard) multiplication, and

$$g'\big(Z^{[1]}\big) = \text{matrix of derivatives of } g \text{ evaluated at each entry of } Z^{[1]}.$$

- If $g = \text{ReLU}$, then

$$g'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \le 0, \end{cases} \quad \Longrightarrow \quad \big[g'(Z^{[1]})\big]_{j,i} = \begin{cases} 1, & Z^{[1]}_{j,i} > 0, \\ 0, & Z^{[1]}_{j,i} \le 0. \end{cases}$$

- If $g = \tanh$, then

$$g'(z) = 1 - \tanh^2(z), \quad \Longrightarrow \quad \big[g'(Z^{[1]})\big]_{j,i} = 1 - \big(A^{[1]}_{j,i}\big)^2.$$

- If $g = \sigma$ (sigmoid), then $g'(z) = \sigma(z)\big(1 - \sigma(z)\big)$.

Thus

$$dZ^{[1]} = dA^{[1]} \circ g'\big(Z^{[1]}\big) \quad \in \mathbb{R}^{n_h \times m}.$$

### 3.2 Gradient w.r.t. $W^{[1]}$

From

$$Z^{[1]} = W^{[1]}\,X + b^{[1]},$$

treating $X$ as constant,

$$\frac{\partial J}{\partial W^{[1]}} = \frac{\partial J}{\partial Z^{[1]}}\,\frac{\partial Z^{[1]}}{\partial W^{[1]}} = \frac{1}{m}\,\big(dZ^{[1]}\big)\,X^T \;\in\; \mathbb{R}^{n_h \times n_x}.$$

Hence

$$dW^{[1]} = \frac{1}{m}\,dZ^{[1]}\,X^T. \tag{10}$$

### 3.3 Gradient w.r.t. $b^{[1]}$

Because $b^{[1]}$ is added to each column of $W^{[1]}X$,

$$\frac{\partial J}{\partial b^{[1]}} = \frac{1}{m}\sum_{i=1}^{m}\big[dZ^{[1]}\big]_{:,\,i} \;\in\; \mathbb{R}^{n_h \times 1}.$$

Thus

$$db^{[1]} = \frac{1}{m}\sum_{i=1}^{m} dZ^{[1]}_{:,\,i}. \tag{11}$$

# 4. Full Overview

Putting it all together for a two-layer network (with a single hidden layer of size $n_h$):

## 4.1 Forward Propagation

$$\begin{cases} Z^{[1]} = W^{[1]}\, X + b^{[1]}, & A^{[1]} = g\big(Z^{[1]}\big), \\ Z^{[2]} = W^{[2]}\, A^{[1]} + b^{[2]}, & A^{[2]} = \mathrm{softmax}\big(Z^{[2]}\big), \end{cases}$$

$$J = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{n_y} Y_{k,i}\, \log\big(A^{[2]}_{k,i}\big).$$

## 4.2 Backward Propagation

**Output layer**

$$dZ^{[2]} = A^{[2]} - Y \qquad \in \mathbb{R}^{n_y \times m}, \tag{12}$$

$$dW^{[2]} = \frac{1}{m}\, dZ^{[2]} \left(A^{[1]}\right)^T \qquad \in \mathbb{R}^{n_y \times n_h}, \tag{13}$$

$$db^{[2]} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[2]}_{:,i} \qquad \in \mathbb{R}^{n_y \times 1}, \tag{14}$$

$$dA^{[1]} = \big(W^{[2]}\big)^T\, dZ^{[2]} \qquad \in \mathbb{R}^{n_h \times m}. \tag{15}$$

**Hidden layer**

$$dZ^{[1]} = dA^{[1]} \circ g'\big(Z^{[1]}\big) \qquad \in \mathbb{R}^{n_h \times m}, \tag{16}$$

$$dW^{[1]} = \frac{1}{m}\, dZ^{[1]}\, X^T \qquad \in \mathbb{R}^{n_h \times n_x}, \tag{17}$$

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[1]}_{:,i} \qquad \in \mathbb{R}^{n_h \times 1}. \tag{18}$$

## 4.3 Parameter Updates (Gradient Descent)

After computing these gradients, update each parameter by

$$W^{[\ell]} \leftarrow W^{[\ell]} - \alpha\, dW^{[\ell]}, \quad b^{[\ell]} \leftarrow b^{[\ell]} - \alpha\, db^{[\ell]}, \quad \ell = 1, 2,$$

where $\alpha$ is the learning rate.

# 5. Generalization to More Layers

If there are $L$ layers, the pattern is:

$$Z^{[\ell]} = W^{[\ell]}\, A^{[\ell-1]} + b^{[\ell]}, \quad A^{[\ell]} = g^{[\ell]}\big(Z^{[\ell]}\big), \quad \ell = 1, \ldots, L,$$

with $A^{[0]} = X$. The final layer $L$ uses softmax (for multiclass) or sigmoid (for binary).

During backpropagation:

Compute $dZ^{[L]}$ from the loss derivative at the output (e.g., $A^{[L]} - Y$).

For $\ell = L, L-1, \ldots, 2$ :

$$dW^{[\ell]} = \frac{1}{m}\, dZ^{[\ell]} \left(A^{[\ell-1]}\right)^T,$$

$$db^{[\ell]} = \frac{1}{m} \sum_{i=1}^{m} dZ^{[\ell]}_{:,i},$$

$$dA^{[\ell-1]} = \big(W^{[\ell]}\big)^T\, dZ^{[\ell]},$$

$$dZ^{[\ell-1]} = dA^{[\ell-1]} \circ g'^{[\ell-1]}\big(Z^{[\ell-1]}\big).$$

Then update each $W^{[\ell]}$, $b^{[\ell]}$ accordingly.

# References

Michael A. Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015.